

Explicación detallada de código NPL

Este código es como una **máquina procesadora de lenguaje**. Imaginamos que tienes una montaña de libros y quieres que una computadora los "entienda", cuente qué palabras son más importantes y limpie el ruido (como los artículos "el", "la", "un") para quedarse solo con la esencia del mensaje.

A continuación, te explico el paso a paso de este proceso, llamado **Pipeline de Procesamiento de Lenguaje Natural (NLP)**.

1. El Concepto: ¿Qué estamos haciendo?

El objetivo es convertir oraciones comunes sobre lenguajes de programación en una **tabla de números** que una computadora pueda analizar matemáticamente. Para llegar ahí, el código sigue estos pasos:

1. **Tokenización:** Cortar la oración en palabras individuales.
 2. **Limpieza (Stopwords):** Quitar palabras que no aportan significado (como "is", "an", "the").
 3. **Lematización:** Llevar cada palabra a su "raíz" (por ejemplo, transformar "running" en "run").
 4. **Análisis Estadístico:** Calcular qué tan importante es cada palabra.
-

2. Los Componentes del Código

Las Herramientas (Imports)

Al principio, vas a ver muchos import. Son como traer una **caja de herramientas** especializada. Estamos usando nltk (la biblioteca líder para lenguaje) y sklearn (para matemáticas avanzadas y estadísticas).

La Limpieza Inteligente (quitarStopwords_eng)

Esta función es como un filtro.

- **Stopwords:** Son palabras "vacías". Si digo "El gato está en el techo", las palabras más importantes son "gato" y "techo". El resto es ruido.
 - **Excepciones:** El código es listo. Dice: "No toques palabras como *Python* o *JavaScript*". Si las pasamos a minúsculas o las recortamos, el análisis pierde precisión técnica.
-

El Detective de Palabras (get_wordnet_pos y lematizar)

No es lo mismo decir "programó" (verbo) que "programador" (sustantivo).

- La función `get_wordnet_pos` actúa como un detective que identifica si la palabra es un verbo, un nombre o un adjetivo.
- Luego, `lematizar` usa esa información para reducir la palabra a su forma base de diccionario.

3. El Corazón del Proceso: El Corpus

El **Corpus** es simplemente tu conjunto de datos. En este caso, son 10 frases que comparan lenguajes como Python, C++, Rust y JavaScript.

El código toma cada frase y la pasa por la "fábrica" que acabamos de construir:

1. La pica en trozos.
2. Le quita el ruido.
3. La reduce a su raíz.

4. El Resultado Final: La Matriz TF-IDF

Aquí es donde ocurre la magia matemática. El código crea una **Matriz TF-IDF**.

- **TF (Frecuencia de Término):** ¿Cuántas veces aparece la palabra en este texto?
- **IDF (Frecuencia Inversa de Documento):** ¿Qué tan "rara" o "especial" es esta palabra en todo el grupo de textos?

¿Por qué es útil? Si la palabra "lenguaje" aparece en todas las frases, no es muy útil para distinguir de qué trata cada una. Pero si "Rust" solo aparece en dos frases sobre seguridad, esa palabra recibe una puntuación más alta de importancia para esos textos específicos.

5. Resumen del Informe Técnico

Etapas	Función en el código	Resultado
Preparación	<code>word_tokenize</code>	Convierte "Python is fast" en ['Python', 'is', 'fast'].

Filtrado	quitar Stopwords eng	Elimina "is", dejando ['Python', 'fast'].
Normalización	lematizar	Asegura que "running" y "runs" se cuenten como "run".
Vectorización	TfidfVectorizer	Convierte las palabras en una tabla de números para que la IA la entienda.
Visualización	frecuencia.plot	Crea un gráfico de barras con las palabras más usadas.

Lo que vas a ver en pantalla:

1. **El texto limpio:** Se verán las frases reducidas a sus palabras clave (ej: ['python', 'interpret', 'high-level', 'language', 'cplus', 'compil', 'low-level']).
2. **Una tabla numérica:** Donde cada fila es una frase y cada columna es una palabra. Los números indican la "relevancia" de esa palabra en esa frase.
3. **Un gráfico:** Una curva que muestra cuáles son las palabras que más se repiten en todo tu estudio.